A Reactive Swarm Controller for the ARGoS Tunnelling Task

Louis Devroye (523920)

Swarm Intelligence, INFO-H-414

Abstract. This report studies a reactive controller for the ARGoS tunnelling project. The implementation combines local repulsion and attraction vectors, a phase-based state machine, and a small set of arena cues to coordinate a swarm of foot-bots without centralized control. Range-and-bearing communication is used to maintain spacing, proximity and light sensing bias motion, and motor-ground readings detect the black target zone. The controller follows a four-state organization: orientation, grouping, tunnel progression, and black-zone motion.

Keywords: swarm robotics · ARGoS · Lua · tunnelling · foot-bot

1 Introduction

The tunnelling task requires a swarm to cross an obstacle-filled arena, reach the target zone on the right, and minimize contamination of that target by obstacles. The score is the number of robots in the target minus the number of obstacles in the target at the end of a run. The project brief further requires multiple runs with different random seeds, boxplots of the final score, and a scalability analysis across swarm sizes.

The controller is written in Lua for the foot-bot platform and uses the local sensing and actuation model described in the course notes for Lua and the foot-bot API [1, 2]. Its design is intentionally reactive: each robot responds to nearby peers, light, floor color, and obstacles rather than relying on a global map or centralized coordination.

As a comparison baseline, the repository also contains a simpler gray-zone escape controller, implemented in the file `gray_escape.lua` (referred to here as the gray-scale baseline). In this baseline, a robot performs a random walk on white floor until a gray or black floor cue is detected; it then stores the escape direction suggested by the proximity field and follows that vector until the floor returns to white. This baseline is intentionally minimal: it does not use the swarm-level grouping or leader-following rules of the main controller, but it provides a direct reference for evaluating how much the main controller improves collective tunnelling performance. The full code is available in the public repository: `https://github.com/ldevroye/swarm_intelligence`.

This follows the same lesson as Conway's Game of Life: simple local rules can generate large, surprising, and structured collective behavior.

2 Problem Definition and Requirements

The arena is a rectangular environment with a white starting area on the left, an obstacle field in the middle, and a black target area on the right. The project brief specifies a 1,000-second run, corresponding to 10,000 ARGoS simulation steps. This horizon is long enough to observe both the initial swarm assembly phase and the later target-entry dynamics.

The available sensors and actuators include proximity, light, motor-ground, range-and-bearing, colored-blob omnidirectional camera, wheels, LEDs, gripper, and turret. The brief emphasizes cooperative behavior, use of swarm robotics principles, repeated runs over 10 seeds, and a scaling study over several swarm sizes. In addition to the final score, the project data file records the number of robots and obstacles in the target at each step.

2.1 Evaluation Criteria

The main quantities to report are:

- final score as a function of swarm size,
- number of robots in the target over time,
- number of obstacles in the target over time,
- variability across the 10 repetitions for each configuration.

3 Controller Overview

The controller is organized as a state machine with four phases: orientation, grouping, tunnel progression, and black-zone motion. The transition logic is driven by local conditions such as the number of neighbors at a target spacing, floor color, and whether the robot has already entered the black target zone.

At a high level, the controller combines three vector sources:

- a Lennard–Jones interaction over range-and-bearing messages,
- a light-seeking vector from the ambient light sensor,
- obstacle repulsion and tangential corrections from proximity readings.

The implementation uses robot LEDs to expose the current phase. The top beacon is green for robots already in the black zone, orange for robots in the tunnel phase, and blue otherwise. This is not merely decorative: the LED state acts as an external cue that neighboring robots can read indirectly through the surrounding visual field, and the same phase information is also transmitted by range-and-bearing messages. In practice, the green beacon functions as a local recruitment signal: robots in the orientation, grouping, or tunnel phases bias their motion toward nearby green beacons, which indicates that a peer has already reached the target area and can serve as a guide for the rest of the swarm. Once a robot is inside the black zone, it is no longer attracted by the beacon

as a destination signal; instead, it remains in the target region and continues its local randomized motion while maintaining boundary avoidance. This allows the swarm to coordinate better although the obstacle avoidance when reaching for a beacon could use a bit of fine tuning.

Per-robot logs are also written to the `logs/` directory, one file per robot (for example `logs/fb1.log`), with a line appended at each simulator step containing state information, neighbor counts, and event markers such as beacon detection. These logs are useful both for debugging and for post-hoc analysis of whether a robot remained in its intended phase or got stuck in a local congestion pattern.

3.1 Orientation Phase

In the initial phase, robots bias motion toward the light while maintaining distance from nearby robots and obstacles. The controller uses the light vector, the Lennard–Jones interaction, and a small obstacle tangential correction to avoid overcrowding at the start. After a fixed number of steps, the robot moves to the grouping phase.

3.2 Grouping Phase

The grouping phase uses the range-and-bearing system to enforce a loose swarm structure. The code computes a Lennard–Jones interaction at the current peer distance and counts neighbors in a target interval around 80 cm. Once enough neighbors are found consistently for a sufficient number of steps, the robot switches to the tunnel phase.

The controller also computes a leader vector from robots that already advanced into later phases. This creates a simple local bootstrap effect: robots that are already closer to the target influence nearby robots to follow them.

3.3 Tunnel Phase

The tunnel phase reduces the target spacing to 60 cm and strengthens the attraction toward robots already in the tunnel. The phase uses a combination of Lennard–Jones interaction, light attraction, and leader following. If an obstacle is directly ahead, the controller adds a sideways correction so the robot can move around the obstacle field instead of pushing straight through it.

This phase is the main navigation behavior of the current implementation. It is designed to help the swarm advance collectively through the clutter while still preserving a coherent formation.

In that sense, the tunnel phase follows the flocking behavior explored in the exercises: the robots remain locally coupled and keep moving as a group rather than as isolated individuals.

3.4 Black-Zone Phase

When enough floor sensors detect black for several consecutive steps, the robot enters the black-zone state. In this state, the controller keeps the robot in the target region and uses a randomized heading segment to avoid getting stuck on the boundary. The beacon is not a steering target for robots already in this state: the black-zone robot continues its local motion inside the target and uses boundary cues and floor readings rather than following peers. The only robots that intentionally run toward a green beacon are robots in earlier phases, which treat the beacon as a local sign that another robot has already reached the black zone and that the swarm should bias its movement in that direction.

The motor-ground readings are central here: they determine when the robot has reached the target and when it should leave the boundary-following motion. This behavior is close to the random-walk exercise: once inside the black zone, the robot follows a randomized walk-like motion with local corrections instead of a precise global plan.

4 Implementation Details

4.1 Sensors and Local Rules

The controller uses the following signals:

- **Proximity sensors**: obstacle repulsion, obstacle tangential motion, and wall/robot blocking.
- **Light sensors**: orientation toward the ambient cue.
- **Motor-ground sensors**: detection of the black target zone.
- **Range-and-bearing**: local communication and Lennard–Jones spacing.
- **Colored-blob camera**: detection of a green beacon emitted by robots already in the black zone.
- **LEDs**: visual phase feedback during simulation.

The wheel command is computed from the target vector angle with a simple proportional steering law. The result is clamped to the allowed wheel speed. This keeps the behavior easy to reason about and consistent with the reactive style encouraged in the project brief.

4.2 State Transitions

The state machine is driven by local counts and thresholds rather than by global knowledge. The most relevant conditions are:

- a minimum number of neighbors in the target spacing interval,
- a minimum duration during which that grouping condition must hold,
- a consecutive black-floor counter before entering the black zone,
- a beacon-based check to bias motion once inside the target region.

In practice, the different states are not sharply separated in behavior because each phase depends on carefully tuned thresholds and on a growing amount of branching in the control logic. This makes the controller harder to maintain as new rules are added.

4.3 Intended Obstacle Handling

The source also contains an obstacle-handling sequence intended for the black-zone phase, when a grabbable obstacle is centered in front of the robot and the cooldown timer has expired. In that case, the robot would approach the obstacle slowly, accumulate front-contact information, close the gripper once alignment is sufficient, and switch the turret to passive mode while carrying the object. It would then move while holding the obstacle, follow the light-directed motion, and release the object after the carry phase or if the sequence must be aborted. This behavior is guarded by checks that stop the sequence if the floor is no longer black or if the controller leaves the black-zone context, preventing the robot from carrying obstacles back into the gray area.

5 Experimental Protocol

The project brief requests 10 runs per configuration with different random seeds. The comparison therefore reports the distribution of final scores for each swarm size, together with the mean number of robots in the target over time and the mean number of obstacles in the target over time.

5.1 Planned Settings

The main controller is compared against the gray-zone escape baseline, which is implemented in `baseline.lua`. The baseline is a reactive reference controller that switches from random exploration on white floor to a directed escape run when gray or black ground is detected. This makes it a useful comparison point for the more structured swarm behavior of the main controller, which relies on local grouping, leader-following, and tunnel-phase transitions.

Table 1. Experimental settings used for the comparison.

Configuration	Value	Notes
Run duration	10,000 steps	Default setting
Seeds per setting	10	Required
Swarm sizes	5, 10, 20, 30	Requested test sizes
Baseline	`baseline.lua`	Gray-scale escape reference controller

The source code for both the main controller and the baseline is available in the public repository at `https://github.com/ldevroye/swarm_intelligence`.

5.2 Data to Report

6 Results

The aggregated results in Table 2 show a clear difference between the main controller and the baseline. The baseline deteriorates strongly as swarm size

Table 2. Aggregated score statistics for the two controllers across the tested swarm sizes. Values are computed from the 10 repetitions recorded in the experiment log.

Controller	Swarm size	Mean score	Std. dev.	Median score
baseline	5	-6.40	3.72	-7.00
baseline	10	-18.10	5.45	-19.50
baseline	20	-43.10	9.59	-45.50
baseline	30	-66.60	12.32	-66.00
main	5	0.78	1.93	2.00
main	10	-1.30	4.10	-2.00
main	20	-12.40	7.93	-12.50
main	30	-21.50	15.33	-22.00

increases, whereas the main controller remains comparatively stable and achieves a positive mean score for the smallest swarm size.

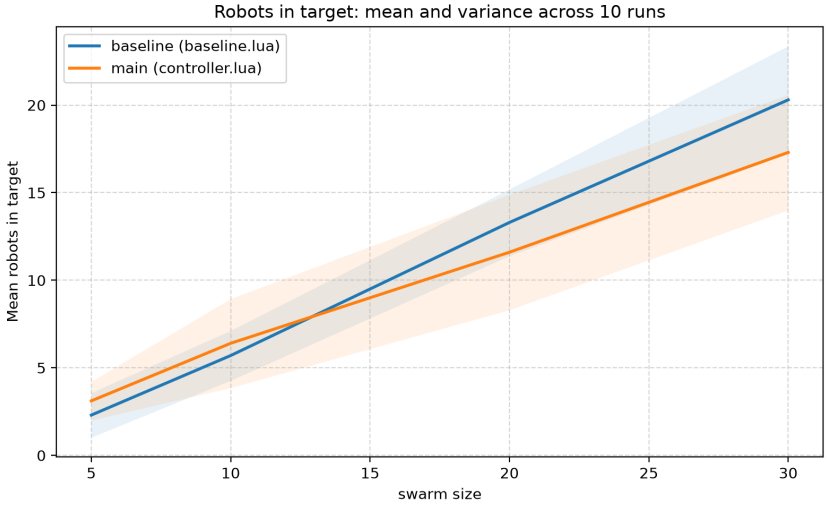

Fig. 1. Mean number of robots in target across the 10 runs, with shaded variance bands for the main controller and the baseline.

6.1 Discussion

The results show that performance does not scale positively with swarm size in this setting. For the main controller, the mean final score is 0.78 at 5 robots, -1.30 at 10 robots, -12.40 at 20 robots, and -21.50 at 30 robots. The trend is therefore not increasing with group size: the best performance is obtained at the smallest swarm, and performance deteriorates as the group becomes denser. The

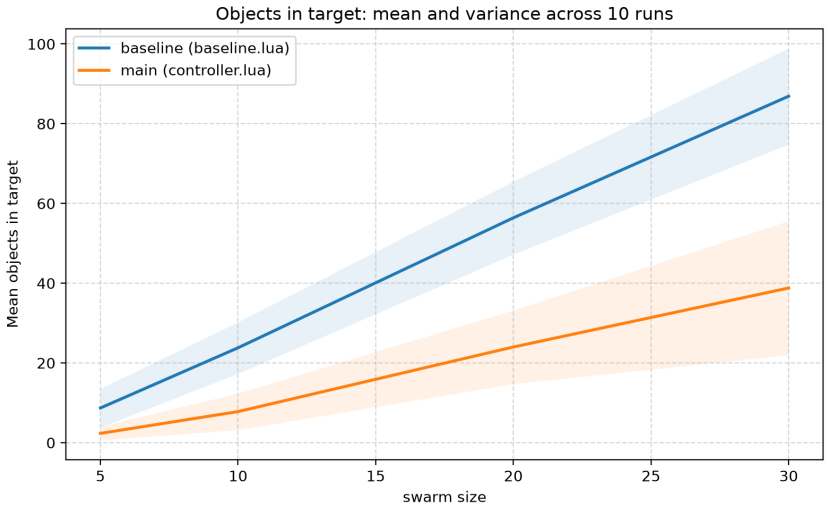

Fig. 2. Mean number of obstacles in target across the 10 runs, with shaded variance bands for the main controller and the baseline.

baseline follows the same pattern but with much lower scores throughout: -6.40, -18.10, -43.10, and -66.60 for 5, 10, 20, and 30 robots, respectively. Hence, the main controller remains superior at every swarm size, but both controllers suffer from congestion as the number of robots grows.

This means that performance is not always increasing. In fact, the observed relationship is closer to a degradation curve: after an initial modest benefit at 5 robots, larger swarms create stronger interference, more blocked paths, and more frequent contact with obstacles. This is consistent with the visual behavior of the main controller: robots increasingly slow down in the black zone and accumulate near the target entrance. The controller does not fully complete the cleanup action that was implemented, and therefore a clogged entrance emerges as robots and obstacles pile up in the same region. Because of this, the output of the swarm is not dominated by uniform progress toward the target but by local congestion at the boundary of the black zone.

The variance also increases significantly with swarm size, especially for the main controller. The standard deviation rises from 1.93 at 5 robots to 15.33 at 30 robots. This indicates that larger groups are more sensitive to timing effects, local collisions, and the specific arrangement of robots near the target entrance. At 20 and 30 robots, the system becomes more unstable, with some runs behaving much better than others depending on whether the target boundary is cleared early enough or whether robots get trapped in a clogged corridor. The baseline also becomes more variable as swarm size grows, but it starts from much less favorable conditions and remains negative in all tested settings.

Several reasons can explain the observed development of the performance. First, the main controller is built around local coordination rules, which work well when the swarm is relatively sparse but become less effective when several robots compete for the same narrow space near the black target zone. Second, once robots enter the target region they slow down substantially, which reduces throughput and makes robot accumulation more likely. Third, the controller does not avoid obstacles sufficiently when a beacon is detected; this causes robots to drag obstacles along the corridor, reinforcing the clogging effect at the target entrance. In other words, the swarm still benefits from local attraction and leader-following, but these corrective mechanisms are not strong enough to prevent local pile-ups under heavier traffic. The result is a system that behaves well in low-density conditions but becomes increasingly bottlenecked as robot density rises.

The baseline, by contrast, is a much simpler reference controller. It is a grayscale escape controller that follows a reactive policy: on white floor it explores or wanders, and when it detects gray or black ground it switches to a directed escape behaviour away from the target region. It does not perform a structured swarm-level formation, no leader following, and no tunnel-building coordination. This makes it a useful baseline because it isolates the effect of the main controller's collective mechanisms, but it also explains why its performance falls sharply as the swarm size increases: it has no mechanism for distributing robots across the corridor or for reducing local congestion near the target. In this sense, the baseline is not a strong competitor in dense conditions, but it provides a consistent lower bound for evaluating how much the cooperative logic of the main controller actually helps.

Overall, the observations support the conclusion that the main controller is qualitatively better than the baseline, but that its current design is limited by congestion in the black zone and by insufficient obstacle avoidance when robots are guided by beacons. These are the main reasons why performance peaks at smaller swarms and then declines as group size increases. Moreover, the obstacle handling setup could drasticly help the score as the obstacles in both removing obstacles from the final zone to unclogging the area for new bots to come in.

References

1. H414 – Swarm Intelligence: Programming the Robots with Lua. Course reference document.
2. H414 – Swarm Intelligence: Sensor and actuator reference for the foot-bot. Course reference document.
3. H414 – Swarm Intelligence: Evaluation and Final Project of INFO-H-414 Swarm Intelligence, Second Session.
4. Springer: Lecture Notes in Computer Science style and sample paper documentation.